

嵌入式面试题集

Linux 系统编程 · 网络编程

Linux 系统编程	22 题 (含答案)
网络编程	18 题 (含答案)
适用岗位	嵌入式 Linux / 服务端 / IoT 后台面试

2026-06-14

Part 1 Linux 系统编程 (22题)

一、进程管理

Q01. 进程和线程的本质区别是什么？

进程是资源分配的基本单位，拥有独立的地址空间、文件描述符表、信号处理等。

线程是 CPU 调度的基本单位，同进程内的线程共享地址空间、文件描述符表，但拥有独立的栈、寄存器上下文、线程本地存储（TLS）。

切换开销：线程切换只需保存寄存器 + 切换栈，成本远低于进程切换（需切换页表 + TLB 刷新）。

★ 同进程线程共享堆、全局变量、FD，但栈独立；进程间地址空间完全隔离

Q02. fork() 调用后父子进程各发生什么？

fork() 使用写时拷贝（Copy-On-Write）：初始时父子进程共享物理页，只有当某方尝试写入时才真正复制该页。

```
pid_t pid = fork();
if (pid < 0) { // fork 失败
    perror("fork"); exit(1);
} else if (pid == 0) { // 子进程 (pid=0)
    execvp(argv[0], argv); // 子进程执行新程序
} else { // 父进程 (pid=子进程PID)
    waitpid(pid, NULL, 0); // 父进程等待子进程退出
}
```

子进程继承：打开的文件描述符、信号处理函数、UID/GID 等。子进程不继承：父进程的内存锁、父进程 PID 自然不同。

★ fork 后立刻 exec 的场景，写时拷贝可以零拷贝完成父进程页面共享

Q03. 僵尸进程和孤儿进程分别是什么？如何避免？

僵尸进程：子进程已退出，但父进程未调用 wait/waitpid 回收，PCB 残留，占用 PID 资源。大量僵尸会耗尽 PID 导致无法创建新进程。

孤儿进程：父进程先于子进程退出，子进程被 init（PID=1）或 systemd 收养，init 会自动调用 wait 回收，无危害。

避免僵尸：方式1 — 调用 waitpid；方式2 — 捕获 SIGCHLD 信号并在处理函数中调用 wait；方式3 — 二次 fork（孙子进程由 init 收养，父进程可直接退出）。

// 信号方式：

```
signal(SIGCHLD, SIG_IGN); // Linux 特性：忽略 SIGCHLD 自动回收子进程
```

Q04. 什么是写时拷贝 (COW)？在哪里体现？

COW (Copy-On-Write)：多个进程/线程共享同一份物理页面，只有当某一方尝试写入时，内核才为写入方复制一份私有副本。

体现场景：fork() 父子进程共享页面；Linux 映射 (mmap MAP_PRIVATE) 读时共享写时复制；Redis 持久化——bgsave fork 子进程遍历数据，父进程写操作触发 COW，保证快照一致性。

二、进程间通信 (IPC)

Q05. Linux IPC 有哪几种方式？各适用什么场景？

管道 (Pipe)：半双工，只能用于亲缘进程；匿名管道 (pipe())，有名管道 (mkfifo)。

消息队列 (POSIX/System V)：有类型的消息，支持优先级，双向，独立于进程生命周期。

共享内存 (shmget/mmap)：最高速 IPC，需配合同步机制 (信号量/互斥锁)。

信号量 (System V semget / POSIX sem)：用于进程/线程间同步和互斥。

套接字 (Socket)：跨网络，最通用，本机通信用 AF_UNIX。

信号 (Signal)：轻量通知机制，携带信息有限 (sigqueue 可携带一个整数)。

D-Bus / Netlink：特定场景 (GUI 进程通信 / 内核-用户空间)。

★ 共享内存速度最快；管道最简单；Socket 最通用 (跨网络跨平台)

Q06. mmap 的 MAP_SHARED 和 MAP_PRIVATE 有何区别？

MAP_SHARED：映射对内存的修改会同步到底层文件，其他映射同一文件的进程可见。

MAP_PRIVATE：修改不影响底层文件，写时拷贝，只在进程私有副本中生效。

典型用途：共享内存 IPC → mmap + MAP_SHARED + shm_open；日志解析 → mmap + MAP_PRIVATE (零拷贝只读)；父子进程共享状态 → mmap + MAP_SHARED + PROT_READ|WRITE，配合 futex 同步。

Q07. POSIX 消息队列与 System V 消息队列的主要区别？

POSIX 消息队列 (mq_open/mq_send/mq_receive)：

· 接口更简洁，类似文件描述符；支持 mq_notify 异步通知；优先级排队。

System V 消息队列 (msgget/msgsnd/msgrcv)：

· 老接口，通过 key 标识；可按消息类型选择性接收；进程退出后消息队列依然存在 (需手动 msgctl IPC_RMID 删除)。

嵌入式 Linux 项目推荐 POSIX 接口，更符合现代 POSIX 标准。

三、POSIX 线程 (pthreads)

Q08. pthread_mutex_lock 和 pthread_spin_lock 的区别？

互斥锁 (mutex) : 竞争失败时线程进入睡眠 (调度出去), 不占 CPU; 适合临界区较长或多核/单核场景。

自旋锁 (spinlock) : 竞争失败时循环忙等, 持续占用 CPU; 适合临界区极短 (微秒级) 且绝对不能睡眠的场景 (如中断上下文用户态模拟)。

嵌入式 Linux 多线程项目: 临界区一般用 mutex; 若临界区只是一个 flag 赋值, 可考虑 GCC 原子操作 (`_sync_xxx` 或 C11 `stdatomic`)。

★ 临界区超过 1~2 条指令就应用 mutex, 自旋锁浪费 CPU

Q09. 什么是死锁? 产生的四个必要条件? 如何预防?

死锁: 两个或多个线程互相等待对方持有的资源, 形成环形等待, 无法继续执行。

四个必要条件 (Coffman 条件) :

1. 互斥: 资源一次只能一个线程使用; 2. 持有并等待: 持有资源的同时请求新资源; 3. 不可抢占: 资源不能被强制剥夺; 4. 循环等待: 线程间形成环形资源依赖。

预防策略: 固定锁的获取顺序 (破坏循环等待); 超时获取 (`pthread_mutex_timedlock`); 资源一次性申请; 死锁检测 (Valgrind Helgrind)。

Q10. pthread_cond_wait 为什么要在 while 循环中使用?

`pthread_cond_wait` 可能发生虚假唤醒 (spurious wakeup) : 即使没有 signal, 线程也可能返回。

用 if 判断条件一次, 虚假唤醒后条件不满足仍会继续执行, 产生错误。

必须用 while 循环:

```
pthread_mutex_lock(&mutex);
while (!condition_ready) { // 虚假唤醒也能被再次检测
    pthread_cond_wait(&cond, &mutex);
}
/* 处理共享数据 */
pthread_mutex_unlock(&mutex);
```

□ POSIX 规范明确允许 `cond_wait` 虚假唤醒, 不用 while 是 bug

Q11. 线程特定数据 (TLS / thread-local storage) 是什么?

TLS 让每个线程拥有同名变量的独立副本, 无需同步即可安全使用。

方式1: POSIX `pthread_key` (`pthread_key_create` / `pthread_setspecific` / `pthread_getspecific`)

方式2: GCC/Clang 扩展 `_thread` 关键字 (C11 用 `_Thread_local`)

```
_thread int t_errno; // 每个线程独立的 errno 副本
_Thread_local int t_count; // C11 标准写法
```

典型场景: `errno` (glibc 本身就是 TLS); 每线程的数据库连接; 日志缓冲区。

四、信号 (Signal)

Q12. signal() 和 sigaction() 的区别?

signal(): POSIX 标准最低要求, 行为在不同平台有差异 (如 handler 执行后是否自动复位为 SIG_DFL) 。

sigaction(): 可靠信号处理, 行为明确:

```
struct sigaction sa;
sa.sa_handler = my_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART; // 被信号打断的系统调用自动重启
sigaction(SIGTERM, &sa, NULL);
```

工程建议: 始终用 sigaction, 不用 signal()。

★ SA_RESTART 能让被信号打断的 read/write 等自动重启, 简化错误处理

Q13. 如何优雅地处理 SIGTERM 信号实现服务程序的平滑退出?

常见模式: 设置 volatile sig_atomic_t 标志, 主循环检测后退出。

```
static volatile sig_atomic_t g_stop = 0;
void sig_handler(int sig) { g_stop = 1; }
int main(void) {
    struct sigaction sa = { .sa_handler = sig_handler };
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);

    while (!g_stop) {
        /* 主循环 */
    }

    // 清理资源、关闭数据库连接
    return 0;
}
```

五、文件 I/O

Q14. read() 返回 0 和 -1 分别意味着什么?

返回 > 0: 实际读取到的字节数 (可能小于请求量, 即短读) 。

返回 0: 到达文件末尾 (EOF) ; 对于 socket 表示对端关闭了写方向 (FIN) 。

返回 -1：出错，需要检查 errno：

```
EINTR -- 被信号中断（通常应重试）
EAGAIN / EWOULDBLOCK -- 非阻塞 fd 上当前无数据
EIO -- 底层 I/O 错误
EBADF -- 无效文件描述符
```

正确循环读取：

```
ssize_t n;
while ((n = read(fd, buf, len)) == -1 && errno == EINTR);
if (n < 0) { perror("read"); }
```

★ read 返回 0 表示 EOF，在网络编程中是对端关闭连接的信号

Q15. epoll 的 ET 和 LT 模式区别？ET 模式编程要注意什么？

LT (Level-Triggered, 水平触发, 默认)：只要缓冲区有数据，epoll_wait 持续返回该 fd。编程简单，不必一次读完。

ET (Edge-Triggered, 边沿触发)：缓冲区从无数据→有数据时才触发一次，必须把数据一次性全部读完，否则下次触发要等新数据到来。

ET 模式编程要点：

```
// ET 模式必须：1.fd 设为非阻塞 2.循环读直到 EAGAIN
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

while (1) {
    ssize_t n = read(fd, buf, sizeof(buf));
    if (n < 0) {
        if (errno == EAGAIN) break; // 数据读完
        perror("read"); break;
    }
    if (n == 0) { /* 对端关闭 */ break; }
    process(buf, n);
}
```

ET 模式效率更高（系统调用次数少），高并发服务器首选，但实现复杂。

★ ET 必须配非阻塞 fd + 循环读完，否则数据丢失

六、内存管理

Q16. malloc 的底层实现是什么？与 mmap 有何关联？

glibc malloc 实现 (ptmalloc2) : 小块内存 (< 128KB) 使用 sbrk/brk 扩展堆 (数据段) ; 大块内存 (>= 128KB) 使用 mmap 申请匿名映射, free 时 munmap 立即归还给 OS。

多线程优化: ptmalloc2 为每个线程维护 per-thread arena, 减少锁竞争; tcmalloc (Google) 和 jemalloc 进一步优化, 高并发服务器常用 tcmalloc/jemalloc 替换。

Q17. 什么是内存泄漏? 如何在 Linux 下检测?

内存泄漏: 申请的堆内存存在不再使用后未被 free, 随时间累积耗尽系统内存。

```
# Valgrind 检测:
valgrind --leak-check=full --track-origins=yes ./your_program

# AddressSanitizer (编译期插桩, 更快):
gcc -fsanitize=address -g your_program.c -o prog
./prog
```

嵌入式 Linux 资源受限场景: 用 /proc/PID/status 中的 VmRSS 监控实际内存使用趋势, 若持续增长则可能有泄漏。

Q18. 用户空间如何实现一个线程安全的内存池?

核心思路: 预分配大块内存, 按固定大小切割成链表; 申请时从链表取头节点; 释放时插回链表头。

线程安全: 给链表操作加 pthread_mutex, 或用无锁的原子 CAS 操作管理链表头。

```
typedef struct Block { struct Block *next; } Block;
Block *pool_head = NULL;
pthread_mutex_t pool_lock = PTHREAD_MUTEX_INITIALIZER;

void *pool_alloc(void) {
    pthread_mutex_lock(&pool_lock);
    Block *b = pool_head;
    if (b) pool_head = b->next;
    pthread_mutex_unlock(&pool_lock);
    return b;
}

void pool_free(void *ptr) {
    Block *b = (Block *)ptr;
    pthread_mutex_lock(&pool_lock);
    b->next = pool_head; pool_head = b;
    pthread_mutex_unlock(&pool_lock);
}
```

七、定时器与守护进程

Q19. Linux 用户态定时器有哪些实现方式？

1. alarm(sec): 粒度为秒, 发送 SIGALRM, 精度低。
2. setitimer: 微秒级, ITIMER_REAL/VIRTUAL/PROF, 发送 SIGALRM/SIGVTALRM/SIGPROF。
3. POSIX timer (timer_create) : 纳秒级; 可用信号或线程通知; 每进程可创建多个定时器。
4. timerfd_create: 将定时器以文件描述符形式暴露, 可配合 epoll 使用, 不需要信号处理:

```
int tfd = timerfd_create(CLOCK_REALTIME, 0);
struct itimerspec its = {
    .it_interval = {0, 10000000}, // 10ms 周期
    .it_value = {0, 10000000},
};
timerfd_settime(tfd, 0, &its, NULL);
// 将 tfd 加入 epoll 监听, 到期时 read(tfd, ...) 返回超时次数
```

高并发服务器推荐 timerfd + epoll, 与 IO 事件统一管理。

★ timerfd 是现代服务器定时器的最佳方案, 与 epoll 无缝集成

Q20. 如何编写一个 Linux 守护进程 (daemon) ?

标准步骤:

```
// 1. fork 使父进程退出 (终端释放子进程)
if (fork() != 0) exit(0);

// 2. 创建新会话, 脱离控制终端
setsid();

// 3. 再次 fork, 确保无法重新获取终端
if (fork() != 0) exit(0);

// 4. 更改工作目录
chdir("/");

// 5. 关闭不需要的文件描述符
close(STDIN_FILENO);
close(STDOUT_FILENO);
close(STDERR_FILENO);

// 6. 重定向标准 I/O 到 /dev/null
open("/dev/null", O_RDONLY); // stdin
open("/dev/null", O_WRONLY); // stdout
open("/dev/null", O_WRONLY); // stderr

// 7. 守护进程逻辑
for (;;) { /* ... */ }
```

现代 Linux 推荐 systemd 服务单元管理守护进程，无需手动 fork 两次。

Q21. select / poll / epoll 的核心区别和复杂度对比？

select: fd 集合用位图 (FD_SET)，最大 1024 个 fd；每次调用需从用户态拷贝 fd 集合到内核；O(n) 扫描；跨平台。

poll: fd 集合为数组，无最大 fd 限制；同样每次调用需全量拷贝；O(n) 扫描；跨平台。

epoll: 事件注册一次 (epoll_ctl)，epoll_wait 只返回就绪 fd；O(1)

获取就绪事件（基于红黑树+就绪链表）；适合大量连接少活跃的场景；Linux 专有。

场景对比 (C=连接数, A=活跃连接数)

select: 每次扫描 C 个 fd, O(C)

poll: 同上, O(C)

epoll: 只处理 A 个就绪 fd, O(A), C 很大 A 很小时优势明显

★ C10K 以上场景必须用 epoll, select/poll 在高连接数下性能急剧下降

Q22. 什么是 Reactor 模式？如何用 epoll 实现一个简单 Reactor？

Reactor 模式：事件循环 (epoll_wait) 监听 IO 事件，就绪时调用对应的处理器 (Handler)。

核心组件：事件循环 (EventLoop) / 事件多路复用器 (Demultiplexer = epoll) / 事件处理器 (Handler)。

简单框架：

```
int epfd = epoll_create1(EPOOL_CLOEXEC);

// 注册 acceptor
struct epoll_event ev = { .events = EPOLLIN, .data.fd = listen_fd };
epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev);

struct epoll_event events[64];
for (;;) {
    int n = epoll_wait(epfd, events, 64, -1);
    for (int i = 0; i < n; i++) {
        if (events[i].data.fd == listen_fd) {
            int conn = accept4(listen_fd, NULL, NULL, SOCK_NONBLOCK);
            ev.events = EPOLLIN | EPOLLET;
            ev.data.fd = conn;
            epoll_ctl(epfd, EPOLL_CTL_ADD, conn, &ev);
        } else {
            handle_read(events[i].data.fd); // 业务处理
        }
    }
}
```

```
}
```

知名实现: libev / libuv / Nginx (单进程多路 IO) / Redis (单线程 AE 事件循环)

Part 2 网络编程（18题）

一、TCP 基础

Q01. TCP 三次握手和四次挥手的详细过程？

三次握手（建立连接）：

```
客户端 --[SYN seq=x]--> 服务端 (SYN_SENT)
客户端 <--[SYN+ACK seq=y ack=x+1]-- 服务端 (SYN_RCVD)
客户端 --[ACK ack=y+1]--> 服务端 (ESTABLISHED)
```

三次握手原因：确保双方各自的收发能力正常（两次不够，一次更不够）。

四次挥手（关闭连接）：

```
主动方 --[FIN]--> 被动方 (FIN_WAIT_1)
主动方 <--[ACK]-- 被动方 (CLOSE_WAIT / FIN_WAIT_2)
主动方 <--[FIN]-- 被动方 (LAST_ACK)
主动方 --[ACK]--> 被动方 (TIME_WAIT 2MSL 后 CLOSED)
```

四次挥手而非三次：因为 FIN 和 ACK 之间可能还有未发完的数据，需要分开发送。

★ 三次握手防半连接攻击；四次挥手因 TCP 半关闭特性不能合并

Q02. TIME_WAIT 状态为什么等待 2MSL？如何减少 TIME_WAIT 对服务器的影响？

2MSL（Maximum Segment Lifetime，通常 1~2分钟）原因：

1. 确保最后一个 ACK 能到达对端（若丢失，对端重发 FIN，2MSL 内能收到）。
2. 确保网络中残留的旧报文消失（避免新连接收到旧数据）。

服务器优化手段：

```
# 启用 SO_REUSEADDR（重用 TIME_WAIT 端口）
int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

# 内核参数调优
net.ipv4.tcp_tw_reuse = 1 # 复用 TIME_WAIT 连接（客户端发起时）
net.ipv4.tcp_fin_timeout = 30 # 减小 FIN 等待时间（默认60s）
```

架构优化：服务端改用长连接（HTTP Keep-Alive）、连接池，减少频繁建立/关闭连接。

★ 服务端不主动 close，由客户端主动关闭，TIME_WAIT 在客户端侧产生

Q03. CLOSE_WAIT 状态大量出现说明什么？

CLOSE_WAIT 说明：对端已发 FIN（四次挥手已完成前两步），本端收到 FIN 后发了 ACK，进入 CLOSE_WAIT 状态，等待本端应用层调用 close()。

大量 CLOSE_WAIT 的根本原因：应用层代码忘记调用 close()

关闭连接，或者处理逻辑卡住（死锁/阻塞）导致迟迟不能 close。

排查方法：

```
# 查看 CLOSE_WAIT 数量
netstat -an | grep CLOSE_WAIT | wc -l

# 找出对应进程
ss -tp | grep CLOSE_WAIT
```

□ CLOSE_WAIT 不会自动消失，必须修复应用层代码，不能靠调内核参数解决

Q04. TCP 粘包和拆包是什么？常用解决方案？

TCP 是字节流协议，无消息边界：

粘包：多个小消息被合并在一次 recv 中返回（Nagle 算法 + 接收缓冲区积累）。

拆包：一个消息被拆成多次 recv（消息超过 MSS，或接收缓冲区不足）。

解决方案：

1. 固定长度消息：每次 recv 固定 N 字节。适合控制帧。
2. 分隔符：以特殊字符（如换行 \n）分隔消息，recv 循环直到找到分隔符。适合文本协议（HTTP 头部）。
3. 长度前缀（TLV）：消息头部携带消息体长度，先读头部，再按长度读体。适合二进制协议（冷链项目串口帧）。

```
// 长度前缀示例
uint16_t msg_len;
recv_all(fd, &msg_len, 2); // 先读 2 字节长度
msg_len = ntohs(msg_len); // 大端转主机字节序
recv_all(fd, buf, msg_len); // 再读消息体
```

★ 嵌入式串口帧通常用固定帧格式+帧头校验+长度字段解决拆包粘包

二、Socket 编程

Q05. TCP 服务器端的完整建立步骤（Socket 到 Accept）？

```
// 1. 创建 socket
int sfd = socket(AF_INET, SOCK_STREAM, 0);

// 2. 设置端口复用
int opt = 1;
```

```
setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// 3. 绑定地址
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;
bind(sfd, (struct sockaddr*)&addr, sizeof(addr));

// 4. 监听 (backlog=128)
listen(sfd, 128);

// 5. 接受连接
struct sockaddr_in cli = {0}; socklen_t len = sizeof(cli);
int cfd = accept(sfd, (struct sockaddr*)&cli, &len);

// cfd 用于与客户端通信, sfd 继续 accept
```

Q06. SO_REUSEADDR 和 SO_REUSEPORT 分别的作用?

SO_REUSEADDR: 允许绑定仍处于 TIME_WAIT

的地址/端口, 服务器重启后无需等待超时。同时允许同一端口绑定不同本地 IP (多网卡场景)。

SO_REUSEPORT (Linux 3.9+) : 允许多个进程/线程绑定相同 IP:Port, 内核负责将连接均衡分发给各 socket。适合多进程/多线程高性能服务器 (Nginx worker 进程模型)。

区别: REUSEADDR 解决 TIME_WAIT 端口占用; REUSEPORT 实现负载均衡, 减少 accept 锁竞争。

Q07. 非阻塞 connect() 的实现步骤?

connect 是异步的, 非阻塞模式下连接建立通过 EPOLLOUT 事件通知:

```
// 1. 创建非阻塞 socket
int fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);

// 2. connect (返回 -1, errno=EINPROGRESS 表示连接中)
int ret = connect(fd, (struct sockaddr*)&addr, sizeof(addr));
if (ret == -1 && errno != EINPROGRESS) { /* 真正错误 */ }

// 3. 注册 EPOLLOUT (连接完成时触发)
ev.events = EPOLLOUT | EPOLLET; ev.data.fd = fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

// 4. EPOLLOUT 触发后检查连接结果
int err; socklen_t len = sizeof(err);
getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &len);
if (err == 0) { /* 连接成功 */ }
else { /* 连接失败, err 为错误码 */ }
```

Q08. 如何实现心跳检测？TCP Keepalive 和应用层心跳的区别？

TCP Keepalive（内核级）：

```
int keepalive = 1;
setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &keepalive, sizeof(int));
int idle = 60; // 空闲 60s 后开始探测
setsockopt(fd, IPPROTO_TCP, TCP_KEEPIDL, &idle, sizeof(int));
int intvl = 10; // 每 10s 发送一次探测包
setsockopt(fd, IPPROTO_TCP, TCP_KEEPINTVL, &intvl, sizeof(int));
int cnt = 3; // 探测 3 次无响应则断开
setsockopt(fd, IPPROTO_TCP, TCP_KEEPCNT, &cnt, sizeof(int));
```

应用层心跳：定期（如每 30s）发送自定义 PING 包，对端回 PONG；超时未收到则主动关闭连接。

区别：TCP Keepalive 在内核层，不能检测应用层假死（进程 hang 但 TCP 正常）；应用层心跳能检测业务层面的连接可用性，更灵活，是 IM / IoT 项目的首选。

★ 物联网 MQTT 协议内置心跳（PINGREQ/PINGRESP），不需要额外实现

三、高并发服务器

Q09. 常见高并发服务器模型有哪些？各有什么优缺点？

多进程（prefork）：稳定，进程崩溃不影响其他；进程间内存隔离，资源重，连接数受限于进程数。Apache 早期模型。

多线程（one thread per connection）：比进程轻，共享内存；线程数过多时上下文切换开销大。

I/O 多路复用（单线程 Reactor）：单线程 + epoll，无锁；适合 I/O 密集；CPU

密集任务会阻塞事件循环。Redis / Nginx 单 worker。

多线程 Reactor（主从 Reactor）：主线程 accept，多个 IO 线程各自 epoll 处理读写；业务线程池处理 CPU 密集。Muduo / Netty 核心模型。

```
主 Reactor: accept() --> 分发 fd 到子 Reactor
子 Reactor (x N): epoll_wait + read/write
线程池: 处理业务逻辑 (DB/文件等)
```

Q10. 什么是 C10K 问题？如何解决？

C10K：单台服务器同时处理 10000 个并发连接的挑战（C = Concurrent，10K = 10000）。

select/poll 的 $O(n)$ 轮询使 n 很大时 CPU 消耗线性增长，无法支撑 C10K。

解决方案：

1. 使用 epoll ($O(1)$ 就绪通知) + 非阻塞 fd + ET 模式。
2. 减少系统调用（批量 send/recv，sendfile 零拷贝）。
3. 减少内存拷贝（用户态协议栈 DPDK，内存池，mmap）。

4. 多核利用：SO_REUSEPORT 多进程，每 CPU 一个 epoll 实例（Nginx worker 模型）。
5. 减少连接数本身：HTTP/2 多路复用，长连接，连接池。

Q11. send() 返回值小于请求发送的字节数，该怎么处理？

send() 是非阻塞发送，内核缓冲区满时会发生短写（partial write），返回实际写入字节数。

错误处理：循环发送直到全部数据写完，或返回 EAGAIN 则注册 EPOLLOUT 等待可写。

```
ssize_t send_all(int fd, const char *buf, size_t len) {
    size_t sent = 0;
    while (sent < len) {
        ssize_t n = send(fd, buf + sent, len - sent, MSG_NOSIGNAL);
        if (n < 0) {
            if (errno == EINTR) continue;
            if (errno == EAGAIN) return sent; // 等 EPOLLOUT 再发
            return -1;
        }
        sent += n;
    }
    return sent;
}
```

MSG_NOSIGNAL：防止对端关闭后 send 触发 SIGPIPE（默认行为是终止进程）。

★ 生产代码必须处理短写，否则高负载下丢失数据

四、UDP 编程

Q12. UDP 和 TCP 的适用场景区分？

TCP：可靠有序，适合文件传输、HTTP、数据库、需要事务的场景。

UDP：低延迟无连接，适合：实时音视频（丢帧比延迟更可接受）；DNS 查询（一来一回，TCP 握手开销大）；物联网广播发现；游戏状态同步；QUIC（基于 UDP 实现可靠传输，比 TCP 更快建立连接）。

嵌入式 IoT 场景：设备发现用 UDP 广播，数据传输视可靠性需求选 TCP 或 MQTT（基于 TCP）。

Q13. 如何在 UDP 上实现可靠传输（如 QUIC 的基本思路）？

UDP 基础上增加：序列号（检测乱序和丢包）；ACK

确认机制；超时重传；滑动窗口（流量控制）；连接多路复用（Connection ID）。

QUIC 特点：0-RTT/1-RTT 握手（TLS 1.3 集成）；无队头阻塞（各流独立重传）；连接迁移（IP 变化不中断连接，以 Connection ID 标识连接）。

嵌入式中简单可靠 UDP 示例思路：


```
// 发送端
packet.seq_id = next_seq++;
sendto(fd, &packet, sizeof(packet), 0, &peer, sizeof(peer));
set_timeout(RETRANSMIT_MS); // 等 ACK

// 接收端 ACK
ack.seq_id = packet.seq_id;
sendto(fd, &ack, sizeof(ack), 0, &sender, sizeof(sender));
```

五、DNS 与 HTTP

Q14. gethostbyname() 和 getaddrinfo() 的区别？为什么推荐后者？

gethostbyname(): 只支持 IPv4，使用全局（非线程安全）的静态缓冲区，已废弃。

getaddrinfo(): 支持 IPv4/IPv6；线程安全；返回链表可遍历所有地址；符合 POSIX 标准，是现代 DNS 解析的标准接口：

```
struct addrinfo hints = {0}, *res, *rp;
hints.ai_family = AF_UNSPEC; // IPv4 或 IPv6
hints.ai_socktype = SOCK_STREAM;
getaddrinfo("example.com", "80", &hints, &res);
for (rp = res; rp != NULL; rp = rp->ai_next) {
    int fd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
    if (connect(fd, rp->ai_addr, rp->ai_addrlen) == 0) break;
    close(fd);
}
freeaddrinfo(res);
```

Q15. HTTP 长连接 (Keep-Alive) 的工作原理？

HTTP 1.0 默认短连接，每次请求后关闭 TCP 连接。

HTTP 1.1 默认长连接 (Connection: keep-alive)：TCP 连接不立即关闭，下一次 HTTP 请求复用同一连接，减少三次握手开销。

服务端控制：通过 Keep-Alive: timeout=N, max=M 限制空闲超时和最大复用次数。

与 HTTP/2 多路复用区别：HTTP/1.1 长连接仍是串行请求；HTTP/2 在同一 TCP 连接上并行发送多个请求，彻底解决 HTTP/1.1 队头阻塞 (HOL blocking)。

嵌入式 IoT 设备与云端通信：推荐 MQTT（基于 TCP 长连接），省去 HTTP 频繁握手开销，适合低功耗设备。